

Machine Learning Review

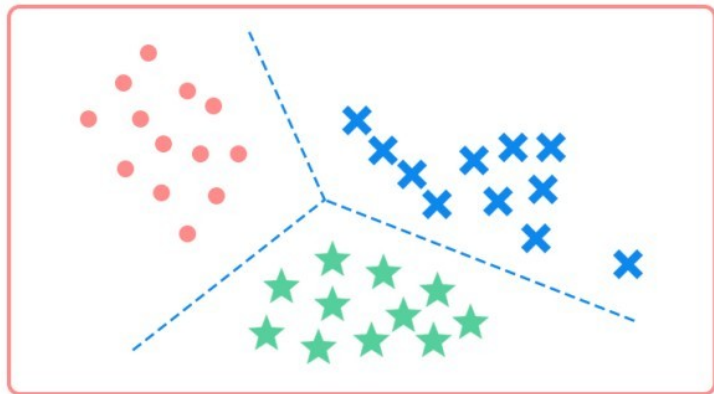
Management Science Principles

Common Types of Machine Learning Problems



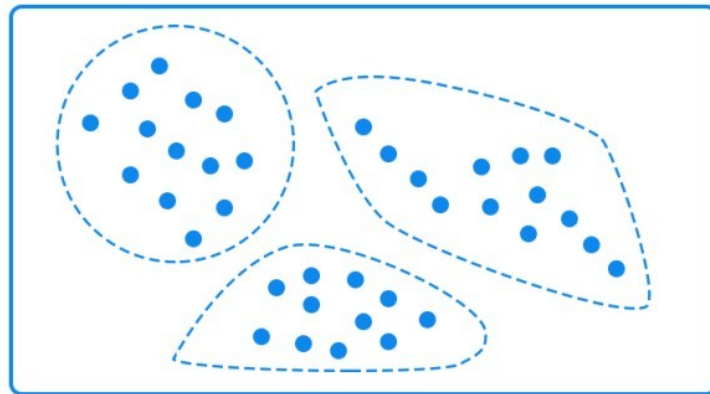
Supervised vs. Unsupervised Learning

Classification



Supervised learning

Clustering



Unsupervised learning

What machine learning algorithm uses this equation?

$$P(y|X) = \frac{P(X|y)P(y)}{P(X)} \quad ; \quad \frac{P(x_1|y)P(x_2|y)\dots P(x_n|y)P(y)}{P(X)}$$

Naïve Bayes!

- Given a feature vector $\mathbf{X}$, we must calculate $P(y|\mathbf{X})$ for all classes y .
 - i.e., from the features, predict the likelihood of each class y .
- Assume independence of all features (the naïve part) to simplify the calculation.

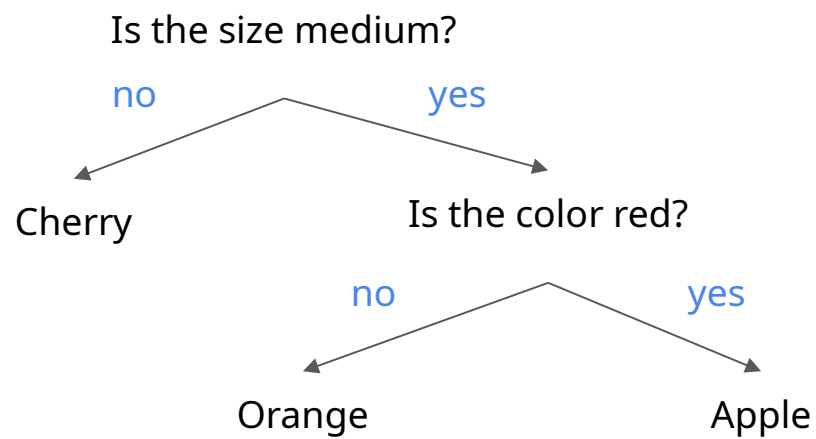
$$P(y|X) = \frac{P(X|y)P(y)}{P(X)} \quad \text{; } \frac{P(x_1|y)P(x_2|y)\dots P(x_n|y)P(y)}{P(X)}$$

Final calculation: $y = \operatorname{argmax}_y \left(P(y) \prod_{i=1}^n P(x_i|y) \right)$

Decision Tree Classifier

- Naïve Bayes uses probabilities from our training data to predict the class of new samples.
- Decision Trees will also use probabilities to model data for classification, but with decision branches.

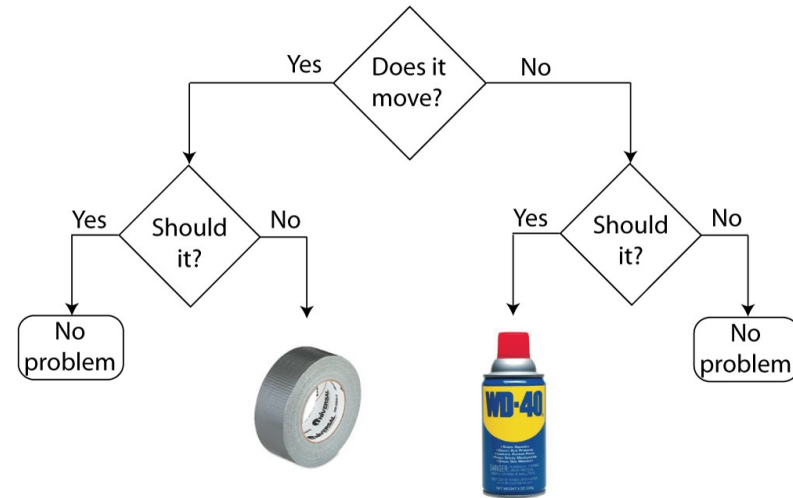
Fruit Name	Color	Size
Apple	Red	Medium
Cherry	Red	Small
Orange	Orange	Medium



Decision Tree Intuition

- Conceptually a bit like branching flow charts.
- Based on feature values, you take certain branches leading to class selections.
- Uses a tree data structure.

Laboratory Troubleshooting Flowchart



Credit: <https://nulab.com>

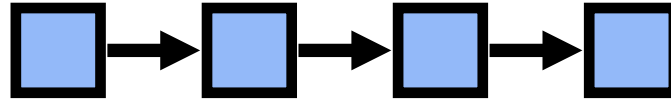
Linear Data Structures

- Recall a few common data structures:

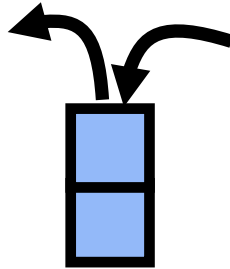
- Arrays



- Linked Lists



- Stacks

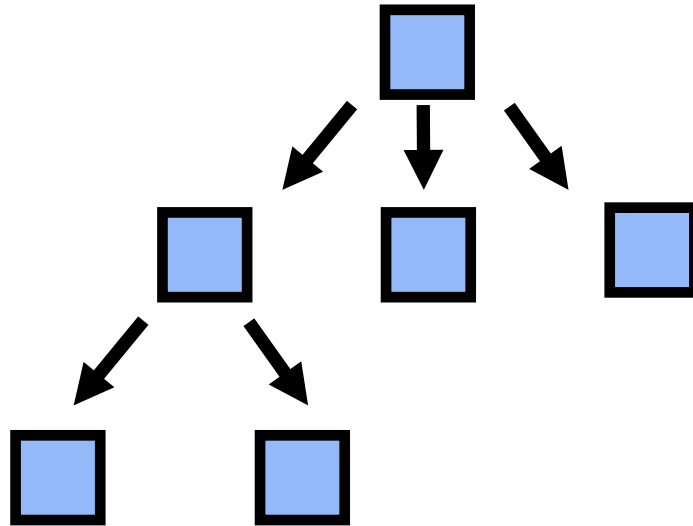


- Queues



Tree Data Structure

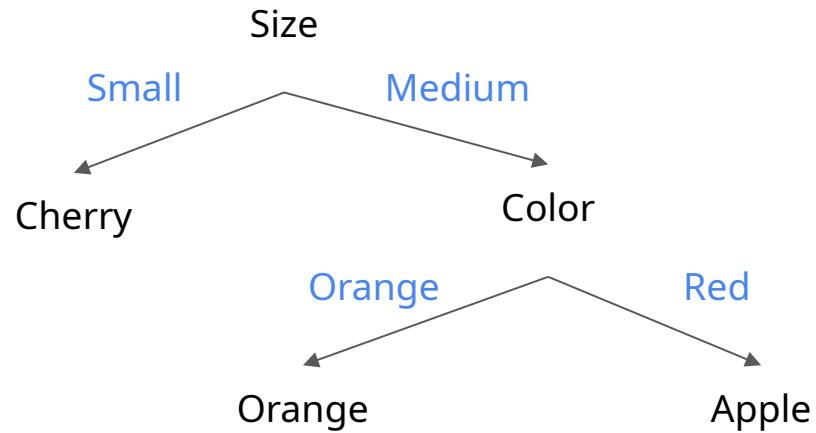
- Trees are a bit like linked lists where each node can point to multiple other nodes.
- This allows branching paths for efficient operations like binary search, heap-based sort, and other algorithms.
- Decision trees as also represented using this structure.



Decision Tree Algorithms

- How to build the tree?
- Popular algorithms include Ross Quinlan's [ID3](#) (Iterative Dichotomiser 3) and its extension C4.5, among others.

Fruit Name	Color	Size
Apple	Red	Medium
Cherry	Red	Small
Orange	Orange	Medium



ID3 High-Level Summary

- The basic idea of this algorithm is as follows:
 - Calculate how “well” each feature can separate the data.
 - Select the feature that “maximally” separates data as the new node.
 - Recursively, split each branch on the remaining features based on how “well” they each perform.
- We need a measure of separability based on features: **entropy**.

Entropy Intuition

- Consider you are about to flip a coin and you want to predict the outcome.
 - If you know the coin is weighted to heads so that it will **always** land on heads, how certain are you of your expected outcome?
 - 100% certain!
 - Entropy is a measure of uncertainty, so let's use that...
 - How uncertain?
 - 0% uncertain

Entropy Intuition

- Now, if you know the coin is fair, then it's a 50/50 heads or tails. How **uncertain** are you now of your outcome?
 - 50% uncertainty???
- Entropy is an interesting measure. It does not care about the raw probabilities of each outcome, only their probabilities relative to each other.
 - i.e., what does the likelihood tell you about the **uncertainty** of the next result?
 - As you may recall from 270, because we only care about the single next result, this is drawn from a Bernoulli distribution!
- In fact, 50% uncertainty would be more like coins weighted 11/89.
- The 50/50 case is actually **100% uncertainty!**

Entropy

- Entropy, H , is the uncertainty of a random variable X as defined by:

$$H(X) = - \sum_{i=1}^n p(x_i) * \log_b(p(x_i))$$

For our coin example:

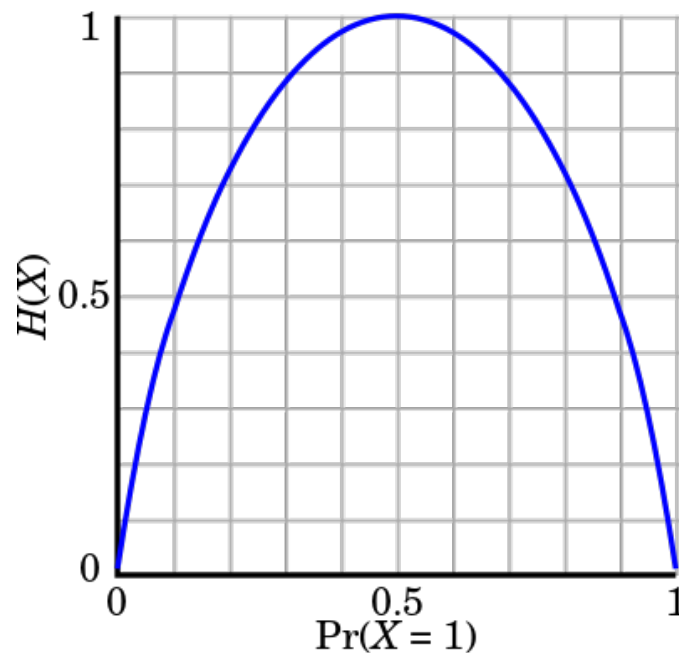
$$H(X) = -p(\text{heads}) * \log_2(p(\text{heads})) - p(\text{tails}) * \log_2(p(\text{tails}))$$

A fair coin:

$$H(X) = -0.5 * \log_2(0.5) - 0.5 * \log_2(0.5) = 1$$

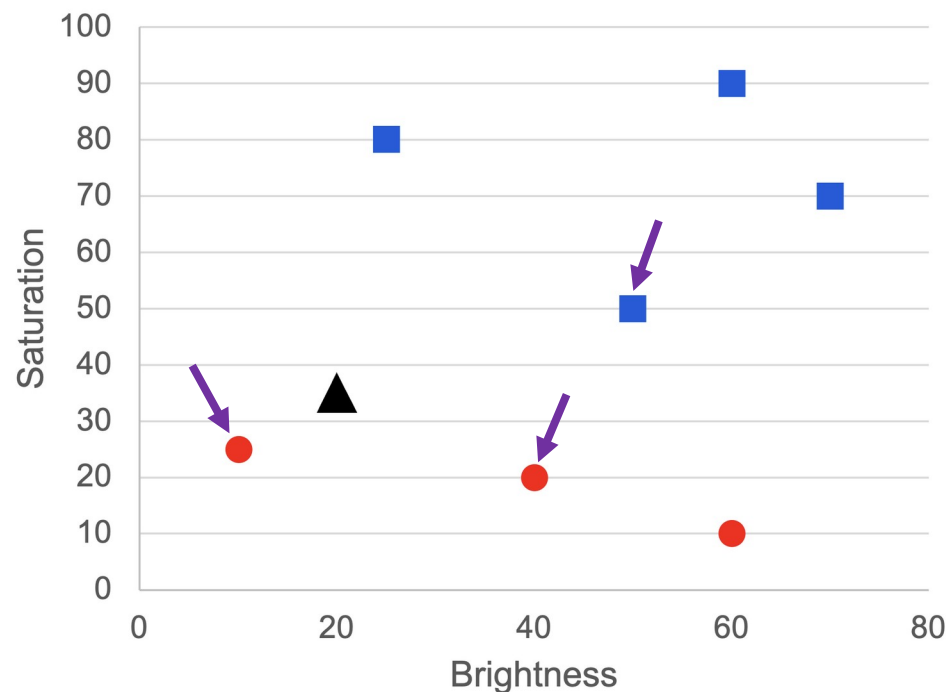
A coin that always lands on heads:

$$H(X) = -1 * \log_2(1) - 0 * \log_2(0) = 0$$

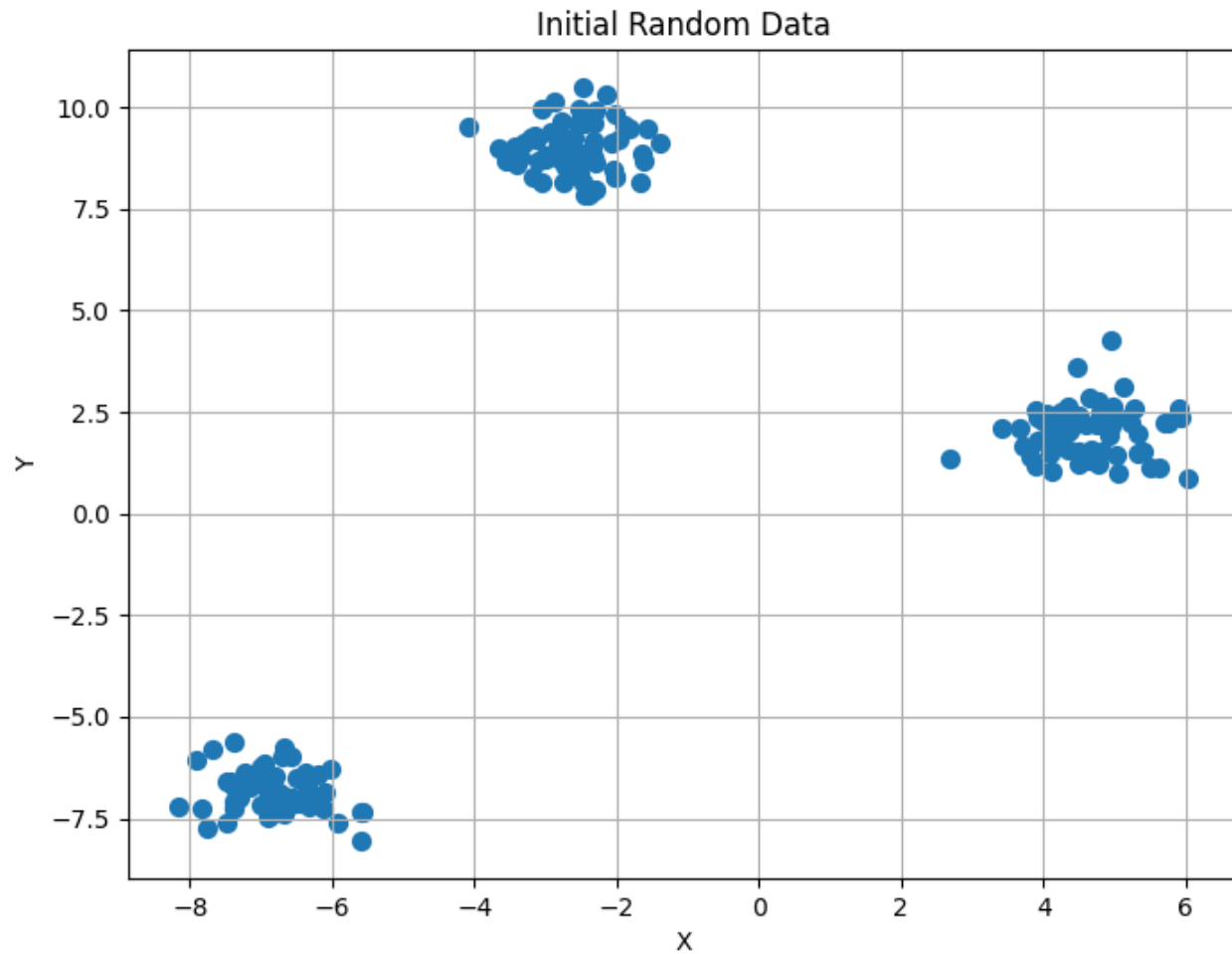


K-Nearest Neighbor

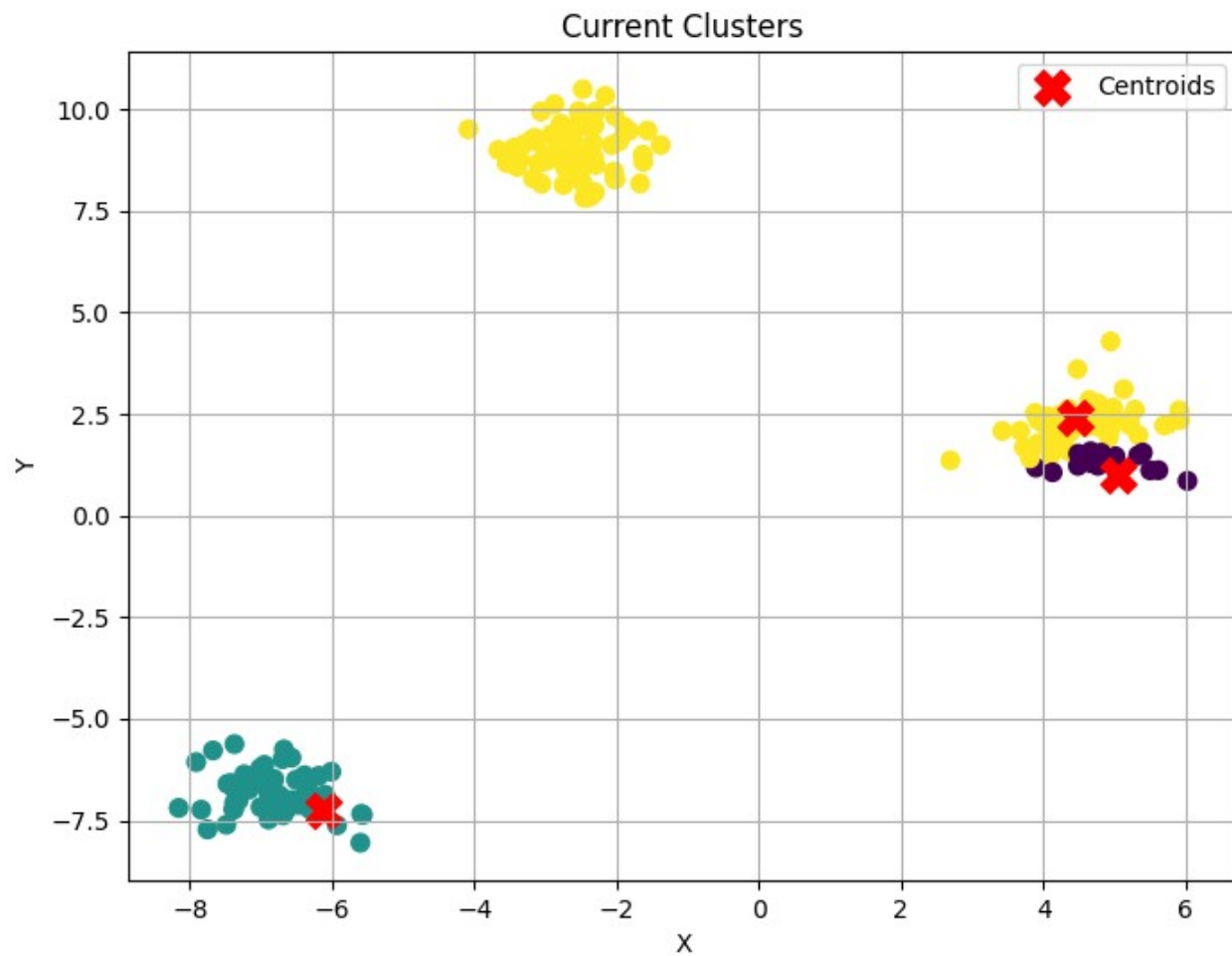
- With KNN, we're just finding the K closest samples and using that label.
- For example, if we have $K=3$, then we take the majority label of the 3 indicated points.
- In this case, we choose Red as the class of the black triangle data point.



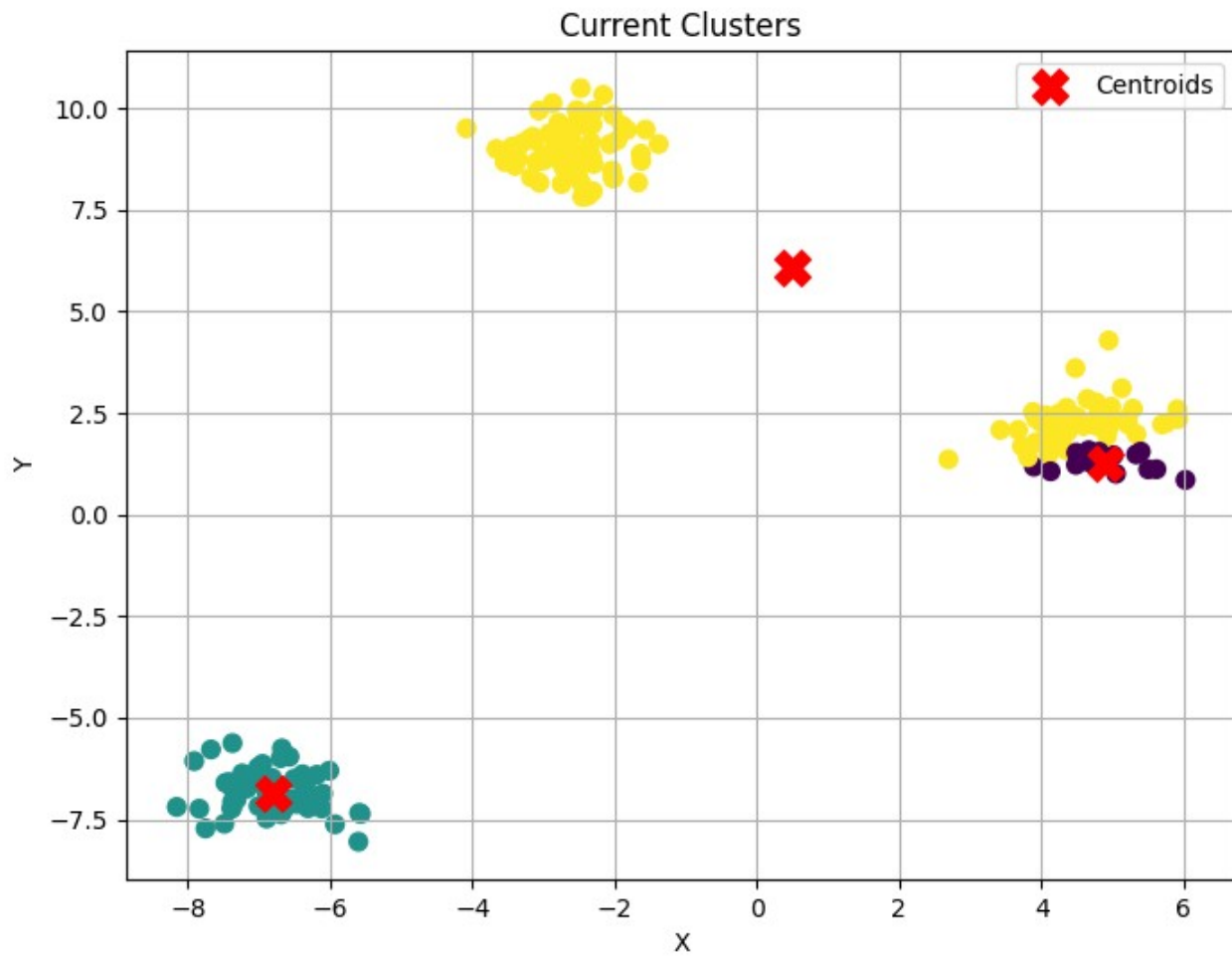
KMeans



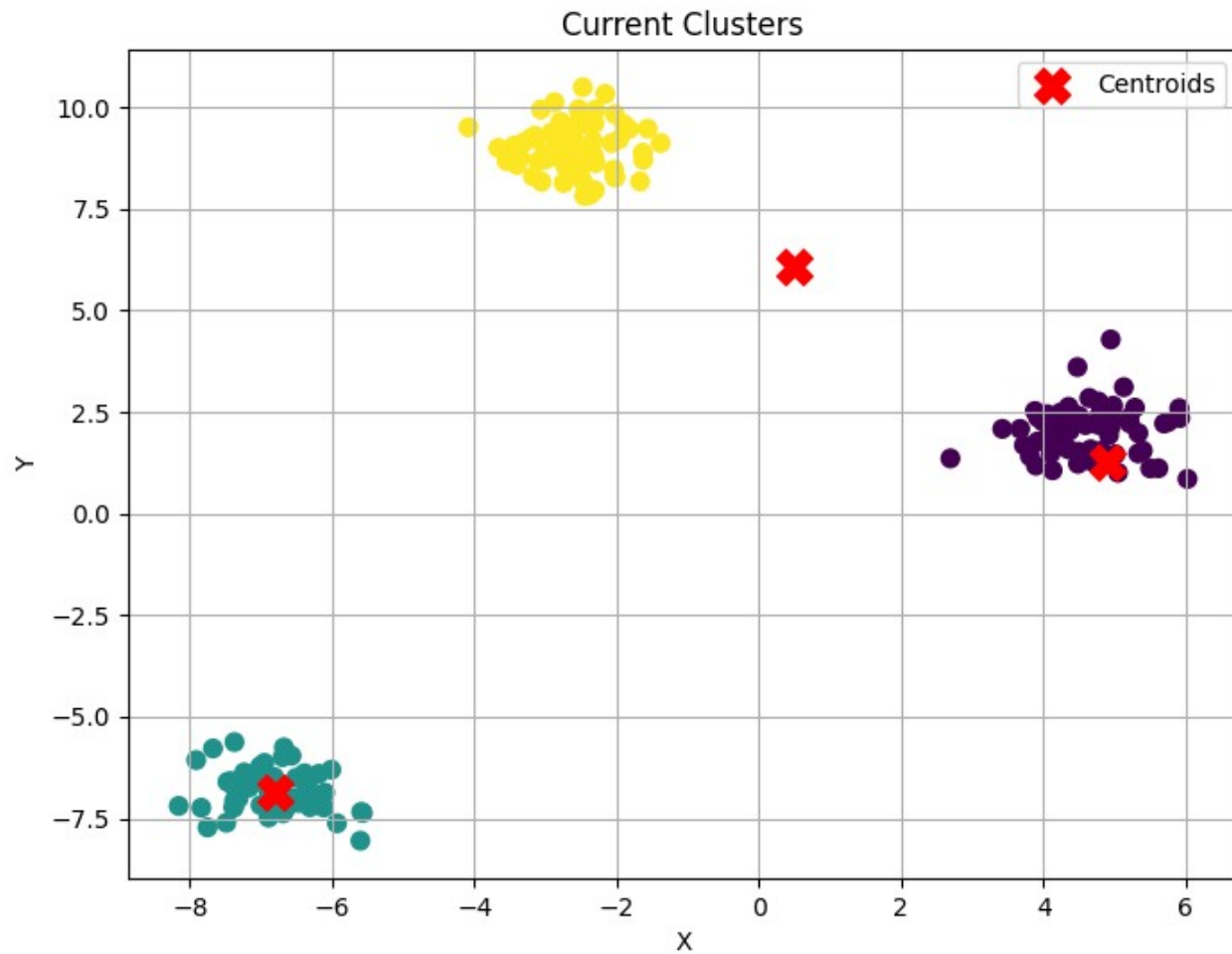
KMeans



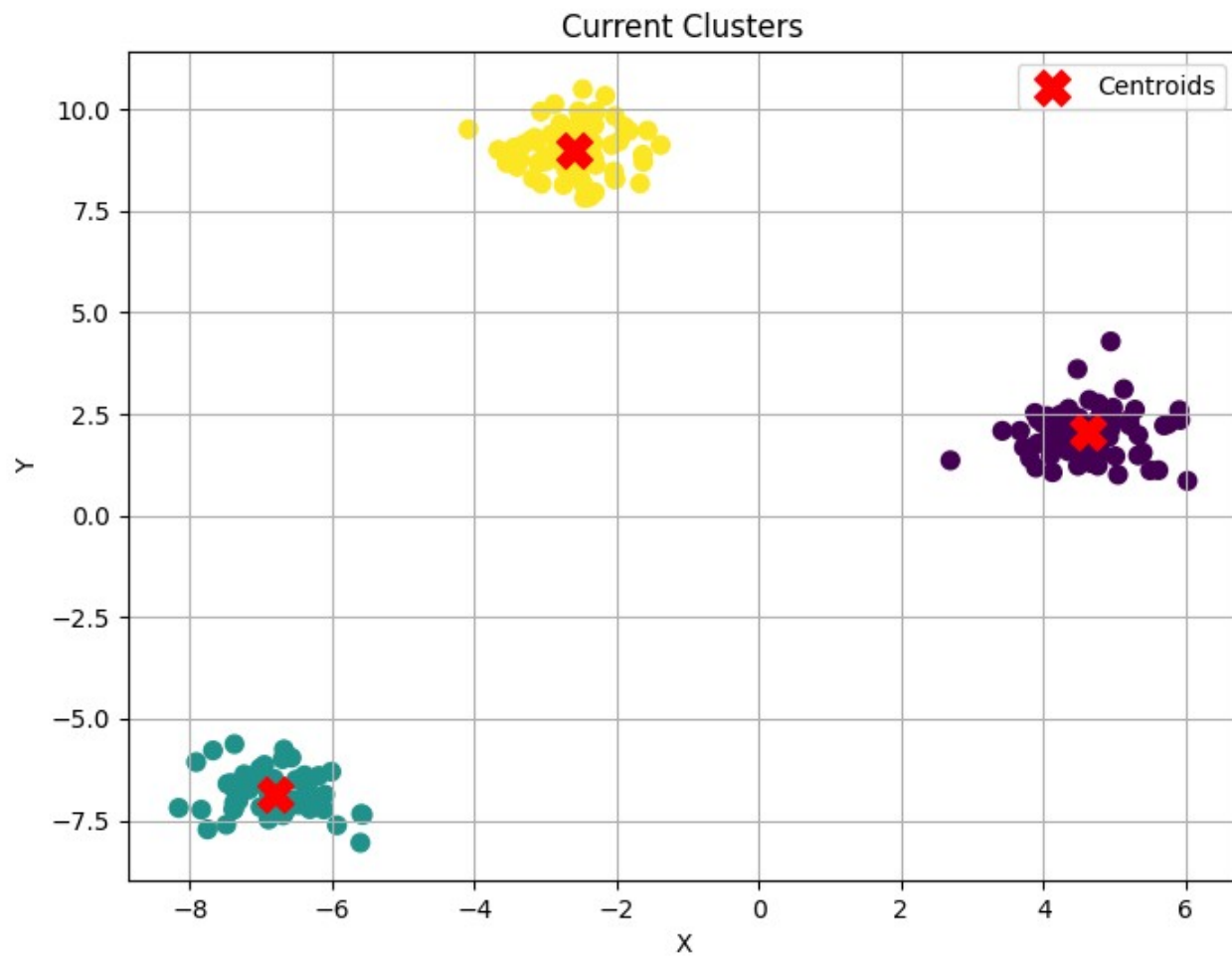
KMeans



KMeans

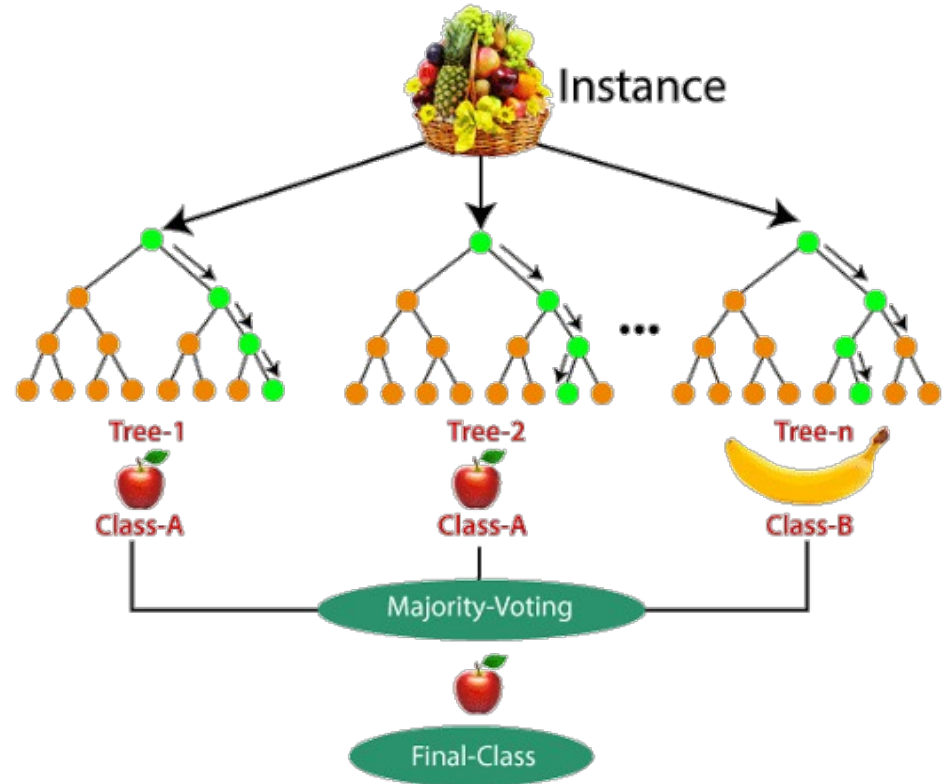


KMeans



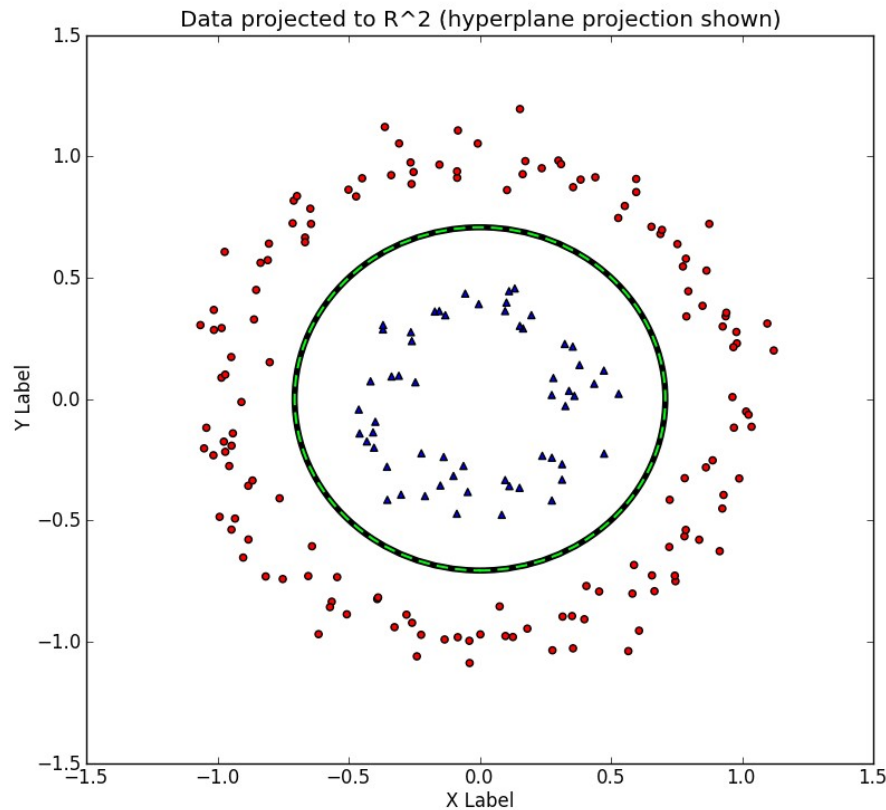
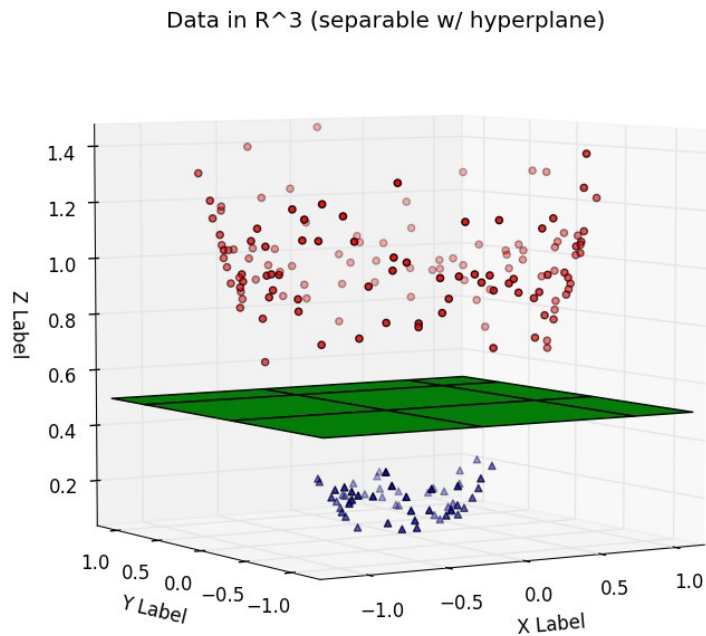
Ensemble methods can combine other classifiers for better results

- For example, random forests use decision trees with **bootstrapped** subsets.
- Train different trees on each subset and combine them!
- This combination of multiple weak learners in parallel is called **bagging**.
- Conversely, **boosting** places weak models in sequence to create a stronger model.

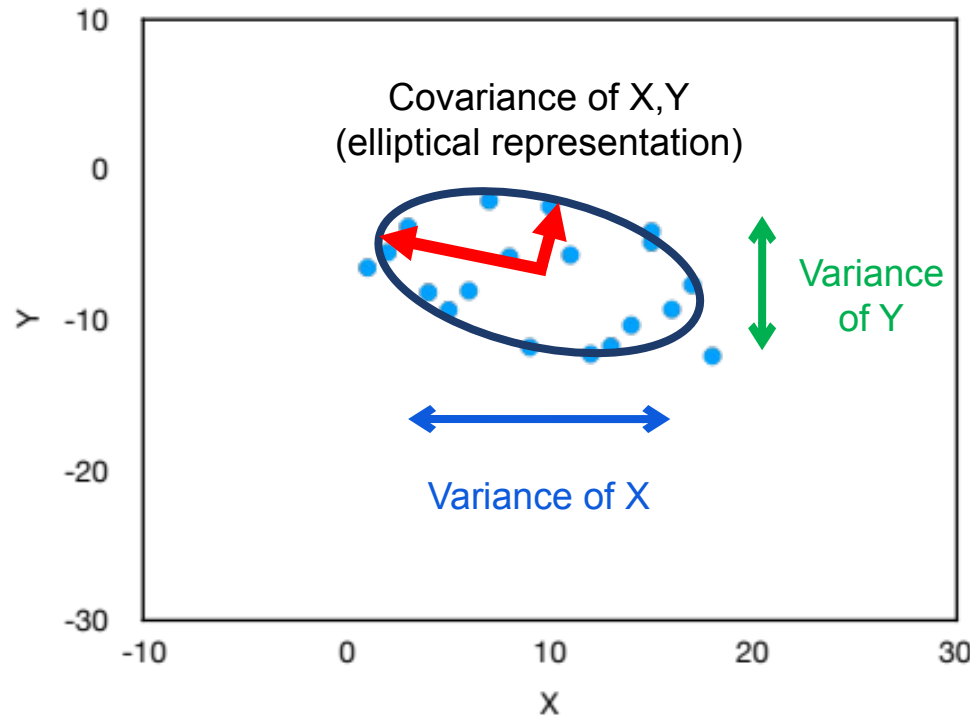


Credit: <https://www.analyticsvidhya.com/blog/2021/06/understanding-random-forest/>

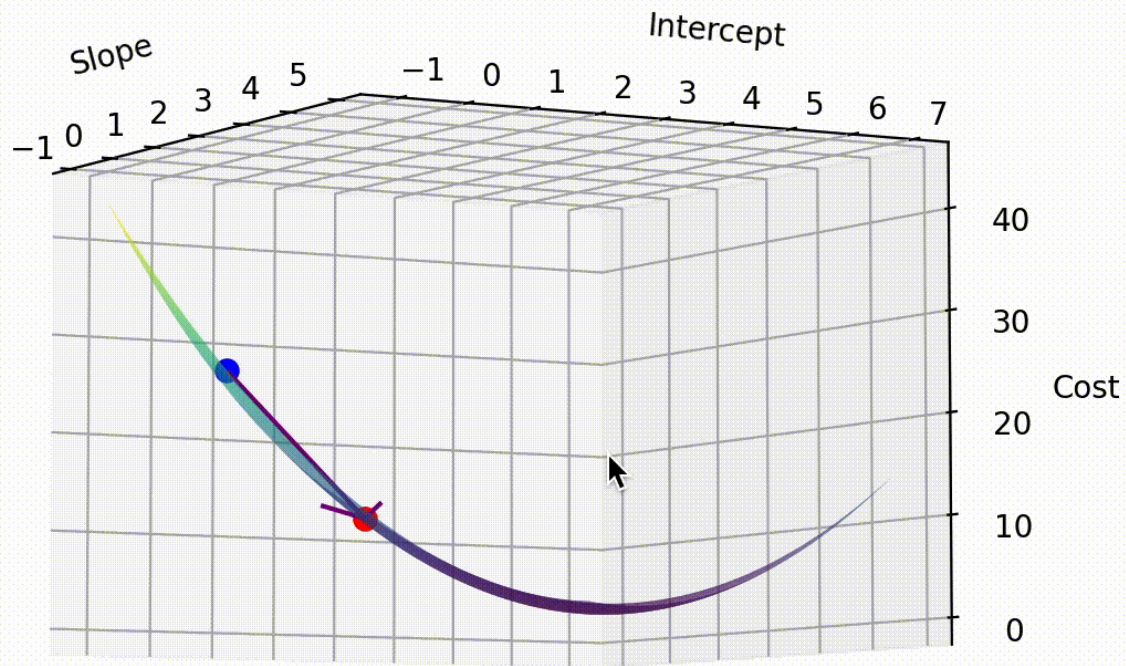
Support Vector Machines use higher dimensional representations to create linear boundaries.



- We also reviewed enough linear algebra to understand matrix representations and components for simple modeling and classification purposes.
- This was useful when we started doing multi-dimensional error minimization via gradient descent. We'll discuss more about optimization in this class!

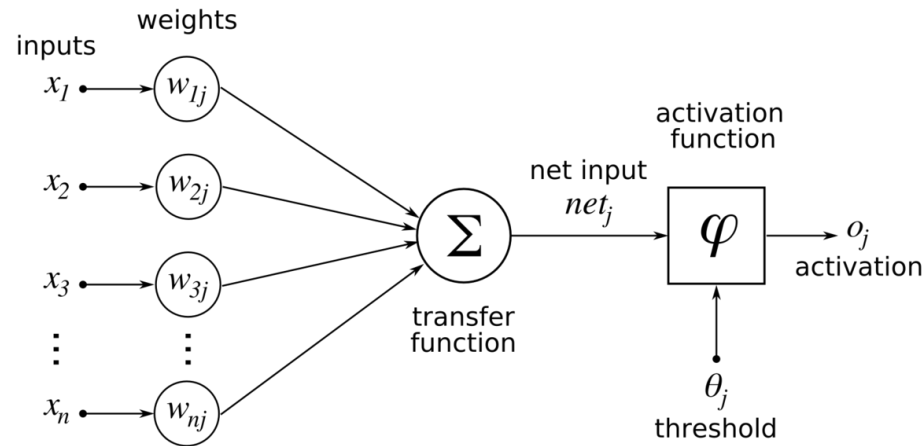


Cost Function Surface



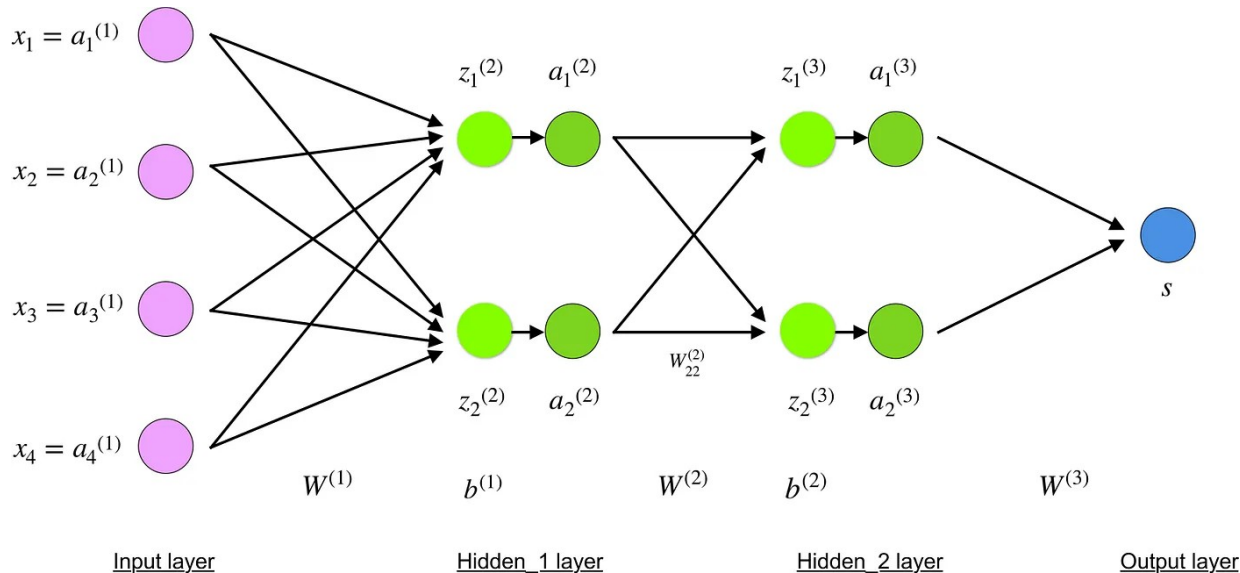
Refresher: Single Perceptron

- Input is fed in (just an array of numbers).
- The weighted sum (a linear equation) is fed into an activation function.
- The output activation gives the final function used for classification.



Multi-Layer Perceptron (MLP) / Neural Network Architecture

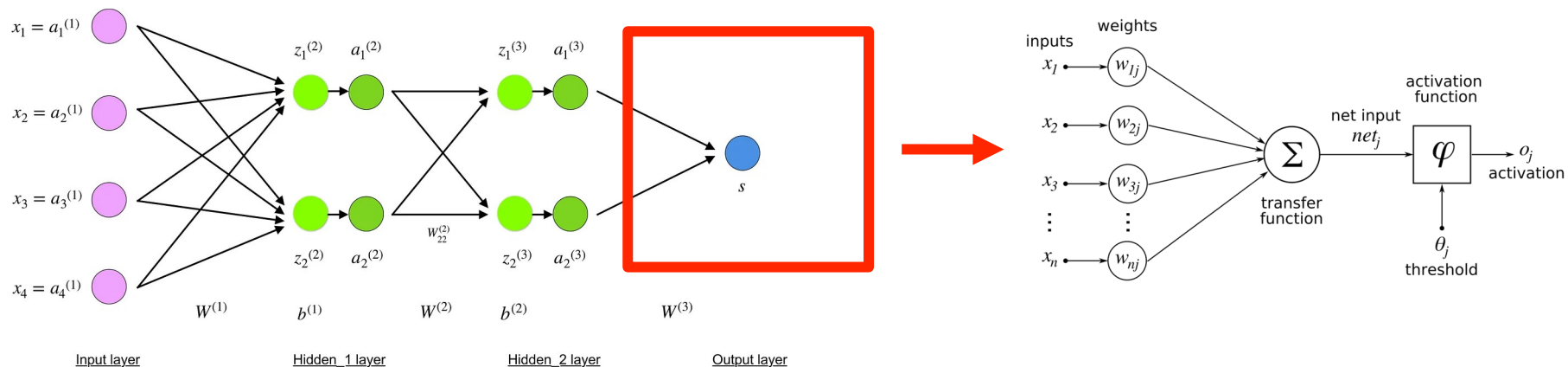
- The MLP is constructed of layers of fully-interconnected perceptrons.
 - Input data is fed into the input layer, just as we would with a single perceptron, now with many.
 - Output activation is read from the output layer, just as with a single perceptron.
 - Between the input and output are neurons comprising the hidden layer(s).



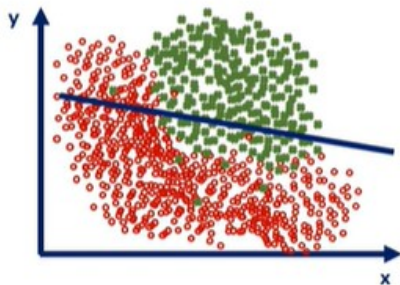
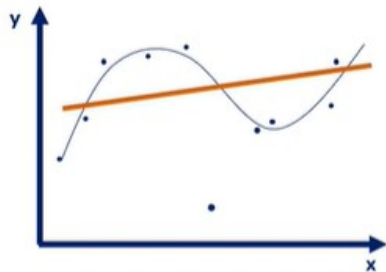
Source: <https://towardsdatascience.com/understanding-backpropagation-algorithm-7bb3aa2f95fd>

MLP Training: Output Layer

- The perceptrons in the output layer behave exactly like any single perceptron.
 - We can compare the MLP output directly against the expected output at this layer, so error calculation and weight updating is unchanged.



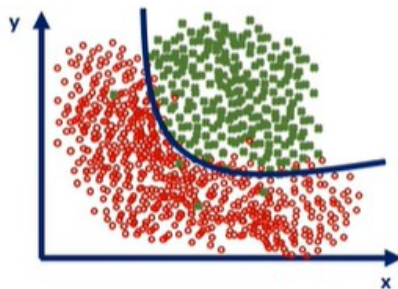
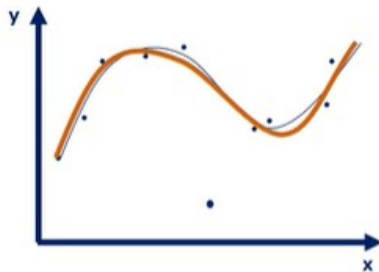
An **underfitted** model



Doesn't capture any logic

- High loss
- Low accuracy

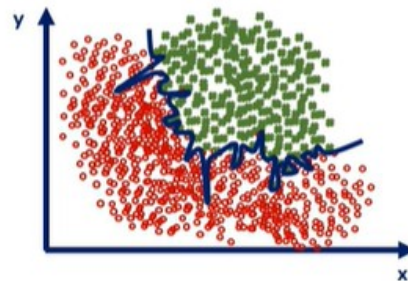
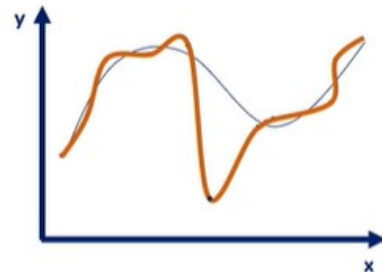
A **good** model



Captures the underlying logic of the dataset

- Low loss
- High accuracy

An **overfitted** model



Captures all the noise, thus "missed the point"

- Low loss
- Low accuracy

Bias-variance tradeoff: The balance between underfitting and overfitting

Confusion Matrices

Let's say you have 20 cats and 20 dogs and 15 pigs, and you have a vision algorithm to recognize them, of the cats it gets 18 right, but classifies 2 as pigs, and of the dogs it gets 16 right, but classifies 3 as pigs and 1 as a cat, and of the pigs, it classifies 5 as cats, 4 as dogs, and the rest correctly.

	Classified as Cat	Classified as Dog	Classified as Pig	Actual Totals
Actually a Cat	18	0	2	20
Actually a Dog	1	16	3	20
Actually a Pig	5	4	6	15
Classifier Totals	24	20	11	55

Single-Class Confusion Matrices

Metrics will use single-class confusion matrices (or averages across single classes).

We need to be able to represent our data as single-class confusion matrices.

	Classified as A	Classified as Not A	
Actually A	Correct (TP)	Wrong (FN)	Total A's
Actually Not A	Wrong (FP)	Correct (TN)	Total Not A's
	Total Classified as A	Total Classified as Not A	

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$TPR = \text{Recall} = \text{Sensitivity} = \frac{TP}{TP + FN}$$

$$FPR = \text{FallOut} = \frac{FP}{FP + TN}$$

$$TNR = \text{Specificity} = \frac{TN}{FP + TN}$$

$$FNR = \text{MissRate} = \frac{FN}{FN + TP}$$

$$\text{Precision} = \frac{TP}{FP + TP}$$

$$F\text{measure} = \frac{2TP}{(FP + 2TP + FN)}$$